

```

        reader.close();
    } catch (IOException e) {
        throw new UnmarshalException(e);
    }
}

public DataObject unmarshal(JsonReader reader, DataObject dto) throws IOException {
    while (reader.hasNext()) {
        String name = reader.nextName();
        setId((ImageLogDTO)dto, reader, name);
    }
    setId((ImageLogDTO)dto);
    return dto;
}

protected void validateId(ImageLogDTO dto) throws IOException {
}

protected void setId(ImageLogDTO dto, JsonReader reader, String name) throws
IOException {
    switch(name) {
        case "guid" :
            {
                if(reader.peek() != JsonToken.NULL) {
                    dto.setGuid(reader.nextString());
                } else {
                    reader.nextNull();
                }
                break;
            }
        case "serviceId" :
            {
                if(reader.peek() != JsonToken.NULL) {
                    dto.setServiceId(reader.nextString());
                } else {
                    reader.nextNull();
                }
                break;
            }
        case "screenId" :
            {
                if(reader.peek() != JsonToken.NULL) {
                    dto.setScreenId(reader.nextString());
                } else {
                    reader.nextNull();
                }
                break;
            }
    }
}

```

```

        case "optreno" :
        {
            if(reader.peek() != JsonToken.NULL) {
                dto.setOptreno(reader.nextString());
            }
            else {
                reader.nextNull();
            }
            break;
        }
        case "clientId" :
        {
            if(reader.peek() != JsonToken.NULL) {
                dto.setClientId(reader.nextString());
            }
            else {
                reader.nextNull();
            }
            break;
        }
        case "requestTime" :
        {
            if(reader.peek() != JsonToken.NULL) {
                dto.setRequestTime(reader.nextString());
            }
            else {
                reader.nextNull();
            }
            break;
        }
        case "responseTime" :
        {
            if(reader.peek() != JsonToken.NULL) {
                dto.setResponseTime(reader.nextString());
            }
            else {
                reader.nextNull();
            }
            break;
        }
        case "exceptionType" :
        {
            if(reader.peek() != JsonToken.NULL) {
                dto.setExceptionType(reader.nextString());
            }
            else {
                reader.nextNull();
            }
            break;
        }
        case "exceptionCode" :
        {
            if(reader.peek() != JsonToken.NULL) {
                dto.setExceptionCode(reader.nextString());
            }
            else {
                reader.nextNull();
            }
            break;
        }
        case "errorMsg" :
        {
            if(reader.peek() != JsonToken.NULL) {
            }
            break;
        }
    }
    if(reader.peek() != JsonToken.NULL) {
    }
}

```

```

        dto.setExceptionMsg( reader.nextString());
    } else {
        reader.nextNull();
    }
    break;
}
default :
    reader.skipValue();
    break;
}
}

@Override
public DataObject unmarshal(Node arg0) throws UnmarshalException {
    return null;
}

@Override
public int unmarshal(byte[] arg0, int arg1, DataObject arg2) throws Exception {
    return 0;
}
}

package nhn.is.fw.common.dto.imageLog;

import com.tmax.superspring.dto.DataObject;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Collections;
import java.util.Map;
import com.tmax.superspring.dto.engine.dto.record.common.FieldProperty;
import java.util.stream.Collectors;

@jakarta.annotation.Generated(
    value = "com.tmaxsoft.sts4.codegen.dto.DtoGenerator",
    date = "26. 7. 28. 오후 4:34",
    comments = "ImageLogDTO"
)
public class ImageLogDTO extends DataObject {
    private static final long serialVersionUID= 1L;

    private String guid = null;

    public String getGuid() {
        return guid;
    }

    public void setGuid(String guid) {
        if(guid == null) {
            this.guid = null;
        }
    }
}

```

```

        }
        public String getServiceId() {
            private String serviceId = null;
        }
        public void setServiceId(String serviceId) {
            if(serviceId == null) {
                this.serviceId = null;
            } else {
                this.serviceId = serviceId;
            }
        }
        public String getScreenId() {
            private String screenId = null;
        }
        public void setScreenId(String screenId) {
            if(screenId == null) {
                this.screenId = null;
            } else {
                this.screenId = screenId;
            }
        }
        public String getOptFno() {
            private String optFno = null;
        }
        public void setOptFno(String optFno) {
            if(optFno == null) {
                this.optFno = null;
            } else {
                this.optFno = optFno;
            }
        }
        public String getClientIp() {
            private String clientIp = null;
        }
        public void setClientIp(String clientIp) {
            return clientIp;
        }
    }

```

```

        }
        public String getExceptionCode() {
            private String exceptionCode = null;
        }
        }
        this.exceptionType = exceptionType;
    } else {
        this.exceptionType = null;
    }
    if(exceptionType == null) {
        public void setExceptionType(String exceptionType) {
    }

    }
    public String getExceptionType() {
        private String exceptionType = null;
    }
    }
    this.responseTime = responseTime;
    } else {
        this.responseTime = null;
    }
    if(responseTime == null) {
        public void setResponseTime(String responseTime) {
    }

    }
    return responseTime;
    }
    public String getResponseTime() {
        private String responseTime = null;
    }
    }
    this.requestTime = requestTime;
    } else {
        this.requestTime = null;
    }
    if(requestTime == null) {
        public void setRequestTime(String requestTime) {
    }

    }
    return requestTime;
    }
    public String getRequestTime() {
        private String requestTime = null;
    }
    }
    this.clientIp = clientIp;
    } else {
        this.clientIp = null;
    }
    if(clientIp == null) {

```

```

        public void setExceptionCode(String exceptionCode) {
            if(exceptionCode == null) {
                this.exceptionCode = null;
            }
            else {
                this.exceptionCode = exceptionCode;
            }
        }

        private String exceptionMsg = null;

        public String getExceptionMsg() {
            return exceptionMsg;
        }

        public void setExceptionMsg(String exceptionMsg) {
            if(exceptionMsg == null) {
                this.exceptionMsg = null;
            }
            else {
                this.exceptionMsg = exceptionMsg;
            }
        }

        public Object clone() {
            ImageLogDTO copyObj = new ImageLogDTO();
            copyObj.clone(this);
            return copyObj;
        }

        public void clone(DataObject _imageLogDTO) {
            if(this == _imageLogDTO)
                return;
            ImageLogDTO _imageLogDTO = (ImageLogDTO) _imageLogDTO;
            this.setGid(_imageLogDTO.getGid());
            this.setServId(_imageLogDTO.getServId());
            this.setScreenId(_imageLogDTO.getScreenId());
            this.setOpterno(_imageLogDTO.getOpterno());
            this.setClntip(_imageLogDTO.getClntip());
            this.setRequestTime(_imageLogDTO.getRequestTime());
            this.setResponseTime(_imageLogDTO.getResponseTime());
            this.setExceptionType(_imageLogDTO.getExceptionType());
            this.setExceptionCode(_imageLogDTO.getExceptionCode());
            this.setExceptionMsg(_imageLogDTO.getExceptionMsg());
        }

        public String toString() {
            StringBuilder builder = new StringBuilder();
            builder.append("gid : ").append(_imageLogDTO.getGid());
            builder.append("servId : ").append(_imageLogDTO.getServId());
            builder.append("screenId : ").append(_imageLogDTO.getScreenId());
            builder.append("opterno : ").append(_imageLogDTO.getOpterno());
            builder.append("clntip : ").append(_imageLogDTO.getClntip());
            builder.append("requestTime : ").append(_imageLogDTO.getRequestTime());
            builder.append("responseTime : ").append(_imageLogDTO.getResponseTime());
            builder.append("exceptionType : ").append(_imageLogDTO.getExceptionType());
        }

```

```

buffer.append("exceptionCode : ").append(exceptionCode).append("\n");
buffer.append("exceptionMsg : ").append(exceptionMsg).append("\n");
return buffer.toString();
}

```

```

private static final Map<String, FieldProperty> fieldPropertyMap;

```

```

static {
    fieldPropertyMap = new java.util.LinkedHashMap<String, FieldProperty>(10);
    fieldPropertyMap.put("guid", fieldProperty.builder()
        .setPhysicalName("guid")
        .setLogicalName("거래고유ID")
        .setType(FieldProperty.Type.STRING)
        .setDecimal(-1)
        .setNullable(true)
        .setEncrypt(false)
        .build());
    fieldPropertyMap.put("serviceId", fieldProperty.builder()
        .setPhysicalName("serviceId")
        .setLogicalName("서비스ID")
        .setType(FieldProperty.Type.STRING)
        .setDecimal(-1)
        .setNullable(true)
        .setEncrypt(false)
        .build());
    fieldPropertyMap.put("screenId", fieldProperty.builder()
        .setPhysicalName("screenId")
        .setLogicalName("화면ID")
        .setType(FieldProperty.Type.STRING)
        .setDecimal(-1)
        .setNullable(true)
        .setEncrypt(false)
        .build());
    fieldPropertyMap.put("optFno", fieldProperty.builder()
        .setPhysicalName("optfno")
        .setLogicalName("옵션아이디")
        .setType(FieldProperty.Type.STRING)
        .setDecimal(-1)
        .setNullable(true)
        .setEncrypt(false)
        .build());
    fieldPropertyMap.put("clientId", fieldProperty.builder()
        .setPhysicalName("clientId")
        .setLogicalName("클라이언트IP")
        .setType(FieldProperty.Type.STRING)
        .setDecimal(-1)
        .setNullable(true)
        .setEncrypt(false)
        .build());
    fieldPropertyMap.put("requestTime", fieldProperty.builder()
        .setPhysicalName("requestTime")
        .setLogicalName("요청시각")
        .setType(FieldProperty.Type.STRING)
        .setDecimal(-1)
        .setNullable(true)
        .build());
}

```

```

import com.tmax.superstring.dto.engine.exception.UmarshalException;
import com.tmax.superstring.dto.engine.exception.MarshallException;
import java.util.ArrayList;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;

package nhis.fw.common.dto.header;

}

}

public Map<String, FieldProperty> getFieldPropertyMap() {
    return Collections.unmodifiableMap(fieldPropertyMap);
}

```

```

        .build();
        .setEncrypt(false)
        .setIsNull(true)
        .setDecimal(-1)
        .setType(FieldProperty.TYPE_OBJECT_STRING)
        .setLogName("응답시간")
        .setPhysicalName("responseTime")
        .build();
        fieldPropertyMap.put("responseTime", fieldProperty.builder()
            .setLogName("응답시간")
            .setPhysicalName("responseTime")
            .build();
        fieldPropertyMap.put("exceptionType", fieldProperty.builder()
            .setLogName("예외종류")
            .setPhysicalName("exceptionType")
            .build();
        fieldPropertyMap.put("exceptionCode", fieldProperty.builder()
            .setLogName("예외코드")
            .setPhysicalName("exceptionCode")
            .build();
        fieldPropertyMap.put("errorMsg", fieldProperty.builder()
            .setLogName("예외메시지")
            .setPhysicalName("errorMsg")
            .build();
        .build();
        .setEncrypt(false)
        .setIsNull(true)
        .setDecimal(-1)
        .setType(FieldProperty.TYPE_OBJECT_STRING)
        .setLogName("예외종류")
        .setPhysicalName("exceptionType")
        .build();
        fieldPropertyMap.put("exceptionType", fieldProperty.builder()
            .setLogName("예외종류")
            .setPhysicalName("exceptionType")
            .build();
        fieldPropertyMap.put("exceptionCode", fieldProperty.builder()
            .setLogName("예외코드")
            .setPhysicalName("exceptionCode")
            .build();
        fieldPropertyMap.put("errorMsg", fieldProperty.builder()
            .setLogName("예외메시지")
            .setPhysicalName("errorMsg")
            .build();
    }
}

```



```

import com.google.gson.stream.JsonReader;
import com.google.gson.stream.JsonWriter;
import com.google.gson.stream.JsonToken;
import org.w3c.dom.Node;

import com.tmax.superspring.dto.engine.base.JsonMessage;

```

```

@jakarta.annotation.Generated(
    value = "com.tmaxsoft.sts4.codegen.message.MessageGenerator",
    date = "26. 7. 21. 오후 2:27",
    comments = "NH N/A 시스템 운용 로그 메시지"
)

```

```

public class SysCommMsgJson extends JsonMessage {
    public byte[] marshal(DataObject obj) throws MarshalException {
        SysComm _sys_comm = (SysComm)obj;
    }
}

```

```

        if (_sys_comm == null)
            return null;

        ByteArrayOutputStream out = null;
        OutputStreamWriter osw = null;
        BufferedWriter bw = null;
        JsonWriter jw = null;
    }
}

```

```

        try {
            out = new ByteArrayOutputStream();
            osw = new OutputStreamWriter(out, this.encoding);
            bw = new BufferedWriter(osw);
            jw = new JsonWriter(bw);
            jw.beginObject();
            marshal(_sys_comm, jw);
            jw.endObject();
            jw.close();
            return out.toByteArray();
        } catch (Exception e) {
            throw new MarshalException(e);
        } finally {
            try {
                if (jw != null)
                    try {
                        jw.close();
                    } catch (IOException e) {
                        throw new MarshalException(e);
                    }
            } finally {
                if (bw != null)
                    try {
                        bw.close();
                    } catch (IOException e) {
                        throw new MarshalException(e);
                    }
            }
        }
    }
}

```

```

        }
    }
}

```

```
public void marshal(Object obj, boolean overwrite, boolean replace)
    throws IOException {
    try {
        // Create the stream
        OutputStream outStream =
            new ObjectOutputStream(
                getOutputStream());
        // Write the object to the stream
        outStream.writeObject(obj);
    } catch (IOException e) {
        // Could not write object.
        throw new IOException(
            "Unable to write object: "
            + e.getMessage());
    }
}
```

```

    }
    finally {
        try {
            if ( osw != null )
                try {
                    osw.close();
                } catch ( IOException e ) {
                    throw new MarshalException(e);
                }
            if ( out != null )
                try {
                    out.close();
                } catch ( IOException e ) {
                    throw new MarshalException(e);
                }
        }
    }
}

```